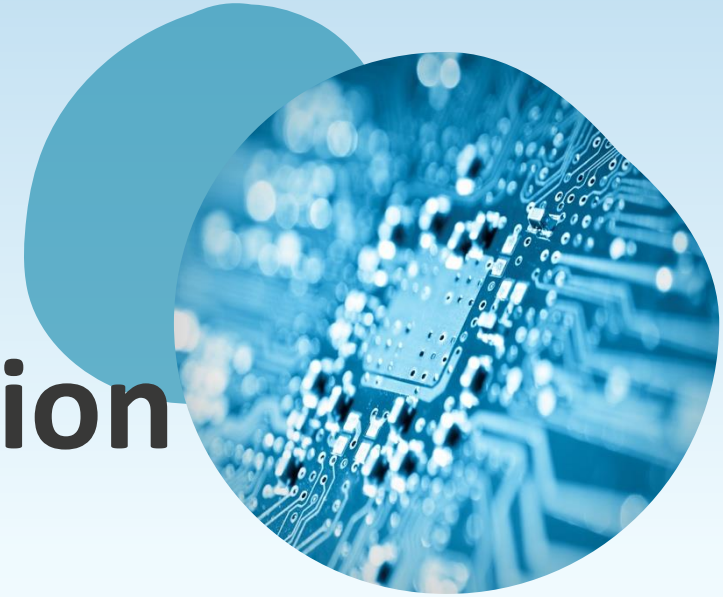
Hardware Abstraction Layer: TI DriverLib



Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

What is DriverLib?

- The Texas Instruments MSP432 Driver Library (DriverLib)
  - A set of fully functional APIs to configure, control, and manipulate the hardware peripherals of the MSP432 platform
- The DriverLib is a **software layer** to the programmer:
  - facilitates higher level of programming compared to direct register accesses
- **Not a layer of intelligence** that provides things automatically!

What is DriverLib?

Traditional Register Access

```
CSKEY = 0x695A;
CSCTL1 |= SELM_1 | DIVM_2;
CSKEY = 0
```



```
CS_initClockSignal(CS_MCLK, CS_VLOCLK_SELECT, CS_CLOCK_DIVIDER_32);
```

DriverLib Equivalent

DriverLib GPIO

```
#include "msp.h"
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

int main(void){
    volatile uint32_t i;

    // Stop watchdog timer
    WDT_A_holdTimer();
    // Set P1.0 to output direction
    GPIO_setAsOutputPin(GPIO_PORT_P1,GPIO_PIN0);

    while(1)     {
        // Toggle P1.0 output
        GPIO_toggleOutputOnPin(GPIO_PORT_P1,GPIO_PIN0);
        // Delay
        for(i=100000; i>0; i--);
    }
}
```

DriverLib GPIO

```
#include "msp.h"
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

int main(void){
    volatile uint32_t i;

    // Stop watchdog timer
    WDT_A_holdTimer();
    // Set P1.0 to output direction
    GPIO_setAsOutputPin(GPIO_PORT_P1,GPIO_PIN0);

    while(1)     {
        // Toggle P1
        GPIO_toggleO
        // Delay
        for(i=100000
    }
}
```

void WDT_A_holdTimer (void)

Holds the Watchdog Timer.

This function stops the watchdog timer from running. This way no interrupt or PUC is asserted.

Returns

None

DriverLib GPIO

```
#include "msp.h"
#include <ti/devices/msp432p4xx/dr

int main(void){
    volatile uint32_t i;

    // Stop watchdog timer
    WDT_A_holdTimer();
    // Set P1.0 to output direction
    GPIO_setAsOutputPin(GPIO_PORT_P

    while(1)     {
        // Toggle P1.0 output
        GPIO_toggleOutputOnPin(GPIO
        // Delay
        for(i=100000; i>0; i--);
    }
}
```

void GPIO_setAsOutputPin (uint_fast8_t *selectedPort*, uint_fast16_t *selectedPins*)

This function configures the selected Pin as output pin.

This function selected pins on a selected port as output pins.

Parameters

selectedPort	is the selected port. Valid values are: ■ GPIO_PORT_P1 ■ GPIO_PORT_P2 ■ GPIO_PORT_P3 ■ GPIO_PORT_P4 ■ GPIO_PORT_P5 ■ GPIO_PORT_P6 ■ GPIO_PORT_P7 ■ GPIO_PORT_P8 ■ GPIO_PORT_P9 ■ GPIO_PORT_P10 ■ GPIO_PORT_P11 ■ GPIO_PORT_PJ
selectedPins	is the specified pin in the selected port. Mask value is the logical OR of any of the following: ■ GPIO_PIN0 ■ GPIO_PIN1 ■ GPIO_PIN2 ■ GPIO_PIN3 ■ GPIO_PIN4 ■ GPIO_PIN5 ■ GPIO_PIN6 ■ GPIO_PIN7 ■ GPIO_PIN8 ■ GPIO_PIN9 ■ GPIO_PIN10 ■ GPIO_PIN11 ■ GPIO_PIN12 ■ GPIO_PIN13 ■ GPIO_PIN14 ■ GPIO_PIN15

Modified bits of **PxDIR** register and bits of **PxSEL** register.

Returns

None

DriverLib GPIO

```
#include "msp.h"
#include <ti/devices/msp432p4xx/driverlib/driverlib.h>

int main(void){
    volatile uint32_t i;

    // Stop watchdog timer
    WDT_A_holdTimer();
    // Set P1.0 to output direction
    GPIO_setAsOutputPin(GPIO_PORT_P1,GPIO_PIN0);

    while(1)     {
        // Toggle P1.0 output
        GPIO_toggleOutputOnPin(GPIO_PORT_P1,GPIO_PIN0);
        // Delay
        for(i=100000; i>0; i--);
    }
}
```

Lab #1: Blinking LEDs

- Modify SingleLED file to blink multiple LEDs on your board.

DriverLib ISR

```
void port6_isr(void){

 uint32_t status = GPIO_getEnabledInterruptStatus(GPIO_PORT_P6);
 GPIO_clearInterruptFlag(GPIO_PORT_P6, status);

 if (status & GPIO_PIN7){
   stateChange = true;
 }
}
```

DriverLib ISR

```
void port6_isr(void){

 uint32_t status = GPIO_getEnabledInte
 GPIO_clearInterruptFlag(GPIO_PORT_P6,

 if (status & GPIO_PIN7){
   stateChange = true;
 }
}
```

uint_fast16_t GPIO_getEnabledInterruptStatus (uint_fast8_t *selectedPort*)

This function gets the interrupt status of the provided PIN and masks it with the interrupts that are actually enabled. This is useful for inside ISRs where the status of only the enabled interrupts needs to be checked.

Parameters

selectedPort	is the selected port. Valid values are: ■ GPIO_PORT_P1 ■ GPIO_PORT_P2 ■ GPIO_PORT_P3 ■ GPIO_PORT_P4 ■ GPIO_PORT_P5 ■ GPIO_PORT_P6 ■ GPIO_PORT_P7 ■ GPIO_PORT_P8 ■ GPIO_PORT_P9 ■ GPIO_PORT_P10 ■ GPIO_PORT_P11 ■ GPIO_PORT_PJ

Returns

Logical OR of any of the following:

- GPIO_PIN0
- GPIO_PIN1
- GPIO_PIN2
- GPIO_PIN3
- GPIO_PIN4
- GPIO_PIN5
- GPIO_PIN6
- GPIO_PIN7
- GPIO_PIN8
- GPIO_PIN9
- GPIO_PIN10
- GPIO_PIN11
- GPIO_PIN12
- GPIO_PIN13
- GPIO_PIN14
- GPIO_PIN15,
- PIN_ALL8,
- PIN_ALL16

indicating the interrupt status of the selected pins [Default: 0]

DriverLib ISR

```
void port6_isr(void){

 uint32_t status = GPIO_getEnable
 GPIO_clearInterruptFlag(GPIO_POR

 if (status & GPIO_PIN7){
   stateChange = true;
 }
}
```

void GPIO_clearInterruptFlag (uint_fast8_t *selectedPort*, uint_fast16_t *selectedPins*)

This function clears the interrupt flag on the selected pin.

This function clears the interrupt flag on the selected pin.

Parameters

selectedPort	is the selected port. Valid values are: ■ **GPIO_PORT_P1** ■ **GPIO_PORT_P2** ■ **GPIO_PORT_PA**
selectedPins	is the specified pin in the selected port. Mask value is the logical OR of any of the following: ■ **GPIO_PIN0** ■ **GPIO_PIN1** ■ **GPIO_PIN2** ■ **GPIO_PIN3** ■ **GPIO_PIN4** ■ **GPIO_PIN5** ■ **GPIO_PIN6** ■ **GPIO_PIN7** ■ **GPIO_PIN8** ■ **GPIO_PIN9** ■ **GPIO_PIN10** ■ **GPIO_PIN11** ■ **GPIO_PIN12** ■ **GPIO_PIN13** ■ **GPIO_PIN14** ■ **GPIO_PIN15**

Modified bits of **PxIFG** register.

Returns
None

Lab #2: GPIO ISR

- Capture button push events and blink different LEDs.

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

void CS_setReferenceOscillatorFrequency (uint8_t *referenceFrequency*)

Selects between the frequency of the internal REFO clock source

Parameters

referenceFrequency	selects between the valid frequencies: ■ **CS_REFO_32KHZ**, ■ **CS_REFO_128KHZ**,

Returns
NONE

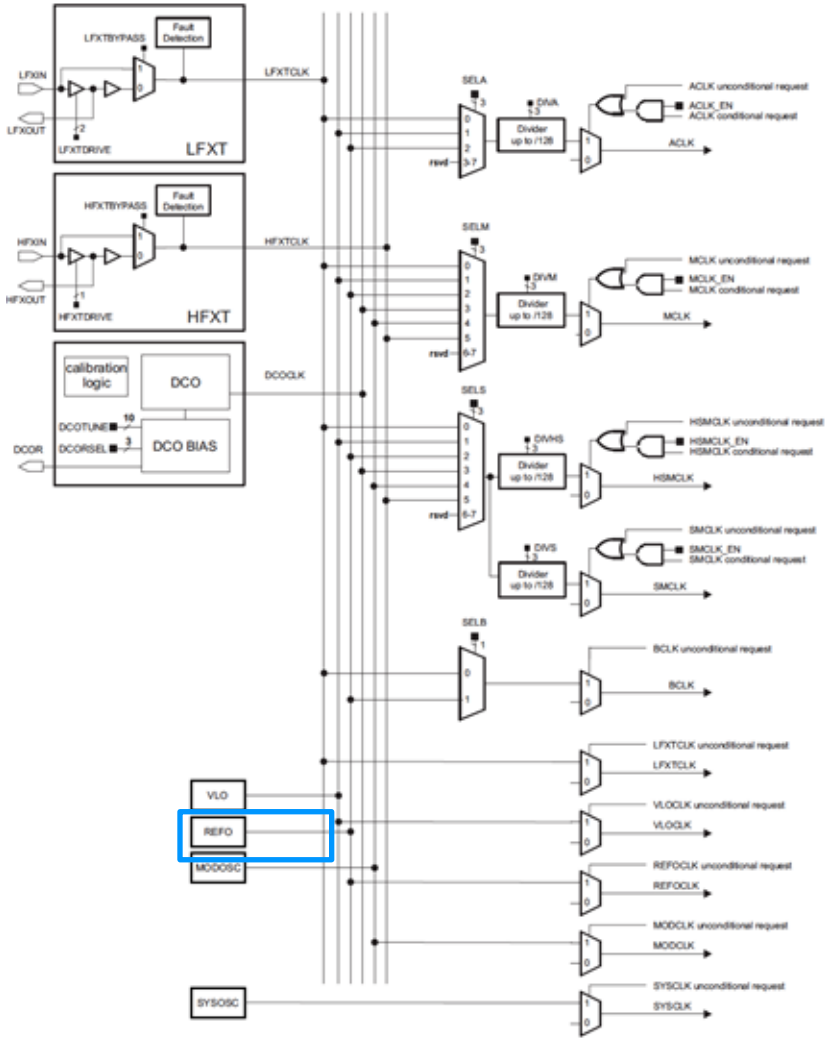


Figure 5-1. Clock System Block Diagram

DriverLib Timers

void CS_initClockSignal (uint32_t *selectedClockSignal*, uint32_t *clockSource*, uint32_t *clockSourceDivider*)

This function initializes each of the clock signals. The user must ensure that this function is called for each clock signal. If not, the default state is assumed for the particular clock signal. Refer to DriverLib documentation for CS module or Device Family User's Guide for details of default clock signal states.

Note that this function is blocking and will wait on the appropriate bit to be set in the CSSTAT READY register to be set before setting the clock source.

Also note that when HSMCLK and SMCLK share the same clock signal. If you change the clock signal for HSMCLK, the clock signal for SMCLK will change also (and vice-versa).

HFXTCLK is not available for BCLK or ACLK.

Parameters

selected-ClockSignal	Clock signal to initialize. ■ **CS_ACLK**, ■ **CS_MCLK**, ■ **CS_HSMCLK** ■ **CS_SMCLK** ■ **CS_BCLK** [clockSourceDivider is ignored for this parameter]
clockSource	Clock source for the selectedClockSignal signal. ■ **CS_LFXTCLK_SELECT**, ■ **CS_HFXTCLK_SELECT**, ■ **CS_VLOCLK_SELECT**, [Not available for BCLK] ■ **CS_DCOCLK_SELECT**, [Not available for ACLK, BCLK] ■ **CS_REFOCLK_SELECT**, ■ **CS_MODOSC_SELECT** [Not available for ACLK, BCLK]
clockSourceDi-vider	- selected the clock divider to calculate clock signal from clock source. This parameter is ignored when setting BLCK. Valid values are: ■ **CS_CLOCK_DIVIDER_1**, ■ **CS_CLOCK_DIVIDER_2**, ■ **CS_CLOCK_DIVIDER_4**, ■ **CS_CLOCK_DIVIDER_8**, ■ **CS_CLOCK_DIVIDER_16**, ■ **CS_CLOCK_DIVIDER_32**, ■ **CS_CLOCK_DIVIDER_64**, ■ **CS_CLOCK_DIVIDER_128**

Returns
NONE

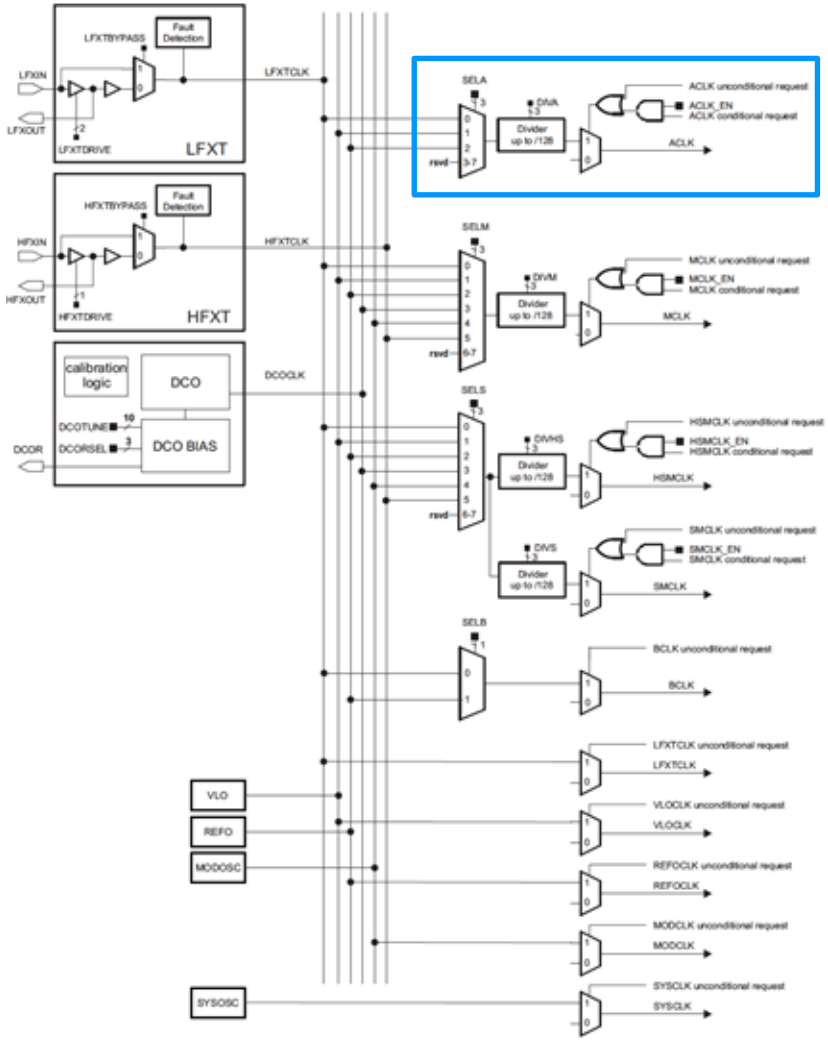


Figure 5-1. Clock System Block Diagram

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

```
/* Starting and enabling ACLK (32k
CS_setReferenceOscillatorFrequency
CS_initClockSignal(CS_ACLK, CS_REF
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

void Timer_A_configureContinuousMode (uint32_t *timer*, const **Timer_A_ContinuousModeConfig** * *config*)

Configures Timer_A in continuous mode.

Parameters

timer	is the instance of the Timer_A module. Valid parameters vary from part to part, but can include: ■ **TIMER_A0_BASE** ■ **TIMER_A1_BASE** ■ **TIMER_A2_BASE** ■ **TIMER_A3_BASE**
config	Configuration structure for Timer_A continuous mode

Configuration options for Timer_A_ContinuousModeConfig structure.

DriverLib Timers

void Timer_A_configureContinuousMode (uint32_t *timer*, const **Timer_A_ContinuousModeConfig** * *config*)

Configures Timer_A in continuous mode.

Parameters

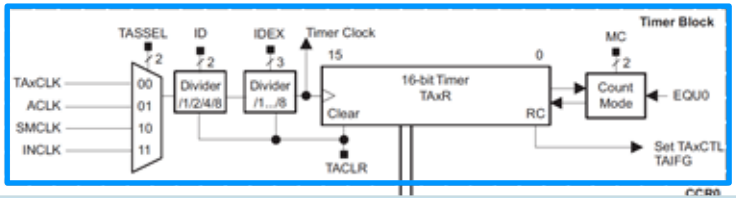
timer	is the instance of the Timer_A module. Valid parameters vary from part to part, but can include: ■ **TIMER_A0_BASE** ■ **TIMER_A1_BASE** ■ **TIMER_A2_BASE** ■ **TIMER_A3_BASE**
config	Configuration structure for Timer_A continuous mode

Configuration options for Timer_A_ContinuousModeConfig structure.

```
/* Starting and enabling ACLK (32k
CS_setReferenceOscillatorFrequency
CS_initClockSignal(CS_ACLK, CS_REF
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_en
/* Starting
Timer_A_star
```

```
const Timer_A_ContinuousModeConfig continuousModeConfig =
{
 TIMER_A_CLOCKSOURCE_ACLK,          // ACLK Clock Source
 TIMER_A_CLOCKSOURCE_DIVIDER_1,     // ACLK/1 = 32.768khz
 TIMER_A_TAIE_INTERRUPT_ENABLE,     // Enable Overflow ISR
 TIMER_A_DO_CLEAR                   // Clear Counter
};
```

DriverLib Timers




```
const Timer_A_ContinuousModeConfig continuousModeConfig =
{
 TIMER_A_CLOCKSOURCE_ACLK,       // ACLK Clock Source
 TIMER_A_CLOCKSOURCE_DIVIDER_1, // ACLK/1 = 32.768khz
 TIMER_A_TAIE_INTERRUPT_ENABLE, // Enable Overflow ISR
 TIMER_A_DO_CLEAR                // Clear Counter
};
```

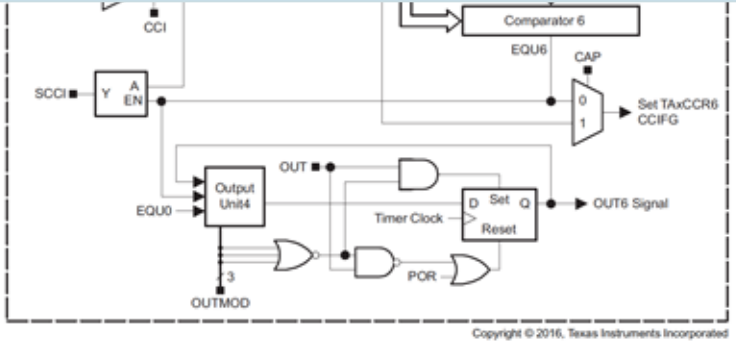


Figure 17-1. Timer_A Block Diagram

Parameters

clockSource	selects Clock source. Valid values are ■ **TIMER_A_CLOCKSOURCE_EXTERNAL_TXCLK** [Default value] ■ **TIMER_A_CLOCKSOURCE_ACLK** ■ **TIMER_A_CLOCKSOURCE_SMCLK** ■ **TIMER_A_CLOCKSOURCE_INVERTED_EXTERNAL_TXCLK**
timerInterruptEnable_TAIE	is the divider for Clock source. Valid values are: ■ **TIMER_A_CLOCKSOURCE_DIVIDER_1** [Default value] ■ **TIMER_A_CLOCKSOURCE_DIVIDER_2** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_4** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_8** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_3** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_5** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_6** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_7** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_10** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_12** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_14** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_16** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_20** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_24** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_28** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_32** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_40** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_48** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_56** ■ **TIMER_A_CLOCKSOURCE_DIVIDER_64**
timerInterruptEnable_TAIE	is to enable or disable Timer_A interrupt. Valid values are ■ **TIMER_A_TAIE_INTERRUPT_ENABLE** ■ **TIMER_A_TAIE_INTERRUPT_DISABLE** [Default value]
timerClear	decides if Timer_A clock divider, count direction, count need to be reset. Valid values are ■ **TIMER_A_DO_CLEAR** ■ **TIMER_A_SKIP_CLEAR** [Default value]

Note

This API does not start the timer. Timer needs to be started when required using the Timer_A_startCounter API.

Returns

None

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_RE
CS_initClockSignal(CS_ACLK, CS_REFOCLK_S
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_
/* Enabling interrupts and going to sleep
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mo
Timer_A_startCounter(TIMER_A0_BASE, TIMER
```

void Interrupt_enableInterrupt (uint32_t *interruptNumber*)

Enables an interrupt.

Parameters

interruptNumber	specifies the interrupt to be enabled.

The specified interrupt is enabled in the interrupt controller. Other enables for the interrupt (such as at the peripheral level) are unaffected by this function.

Valid values will vary from part to part, so it is important to check the device specific datasheet, however for MSP432 101 the following values can be provided:

- FAULT_NMI
- FAULT_HARD
- FAULT_MPU
- FAULT_BUS
- FAULT_USAGE
- FAULT_SVCALL
- FAULT_DEBUG
- FAULT_PENDSV
- FAULT_SYSTICK
- INT_PSS
- INT_CS
- INT_PCM
- INT_WDT_A
- INT_FPU
- INT_FLCTL
- INT_COMP0
- INT_COMP1
- INT_TA0_0
- INT_TA0_N
- INT_TA1_0
- INT_TA1_N
- INT_TA2_0
- INT_TA2_N
- INT_TA3_0
- INT_TA3_N
- INT_EUSCIA0
- INT_EUSCIA1
- INT_EUSCIA2

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

```
/* Starting and enabling
CS_setReferenceOscillator
CS_initClockSignal(CS_ACI
/* Configuring Continuous
Timer_A_configureContinuo
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

bool Interrupt_enableMaster (void)

Enables the processor interrupt.

This function allows the processor to respond to interrupts. This function does not affect the set of interrupts enabled in the interrupt controller; it just gates the single interrupt from the controller to the processor.

Returns

Returns **true** if interrupts were disabled when the function was called or **false** if they were initially enabled.

DriverLib Timers

```
/* Starting and enabling ACLK (32kHz) */
CS_setReferenceOscillatorFrequency(CS_REFO_128KHZ);
CS_initClockSignal(CS_ACLK, CS_REFOCLK_SELECT,CS_CLOCK_DIVIDER_4);
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TIMER_A0_BASE, &continuousModeConfig);
/* Enabling interrupts and going to sleep */
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_N);
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

DriverLib Timers

```
/* Starting and enabling ACLK (32k
CS_setReferenceOscillatorFrequency
CS_initClockSignal(CS_ACLK, CS_REF
/* Configuring Continuous Mode */
Timer_A_configureContinuousMode(TI
/* Enabling interrupts and going t
Interrupt_enableSleepOnIsrExit();
Interrupt_enableInterrupt(INT_TA0_
/* Enabling MASTER interrupts */
Interrupt_enableMaster();
/* Starting the Timer_A0 in continuous mode */
Timer_A_startCounter(TIMER_A0_BASE, TIMER_A_CONTINUOUS_MODE);
```

void Timer_A_startCounter (uint32_t *timer*, uint_fast16_t *timerMode*)

Starts Timer_A counter

Parameters

timer	is the instance of the Timer_A module. Valid parameters vary from part to part, but can include: ■ **TIMER_A0_BASE** ■ **TIMER_A1_BASE** ■ **TIMER_A2_BASE** ■ **TIMER_A3_BASE**
timerMode	selects Clock source. Valid values are ■ **TIMER_A_CONTINUOUS_MODE** [Default value] ■ **TIMER_A_UPDOWN_MODE** ■ **TIMER_A_UP_MODE**

Note

This function assumes that the timer has been previously configured using Timer_A_configureContinuousMode, Timer_A_configureUpMode or Timer_A_configureUpDownMode.

DriverLib Timers

- Two interrupt vectors are associated with the 16-bit Timer_A module:
  - **TAxCCR0** interrupt vector for TAxCCR0 CCIFG
  - **TAxIV** interrupt vector for all other CCIFG flags and TAIFG

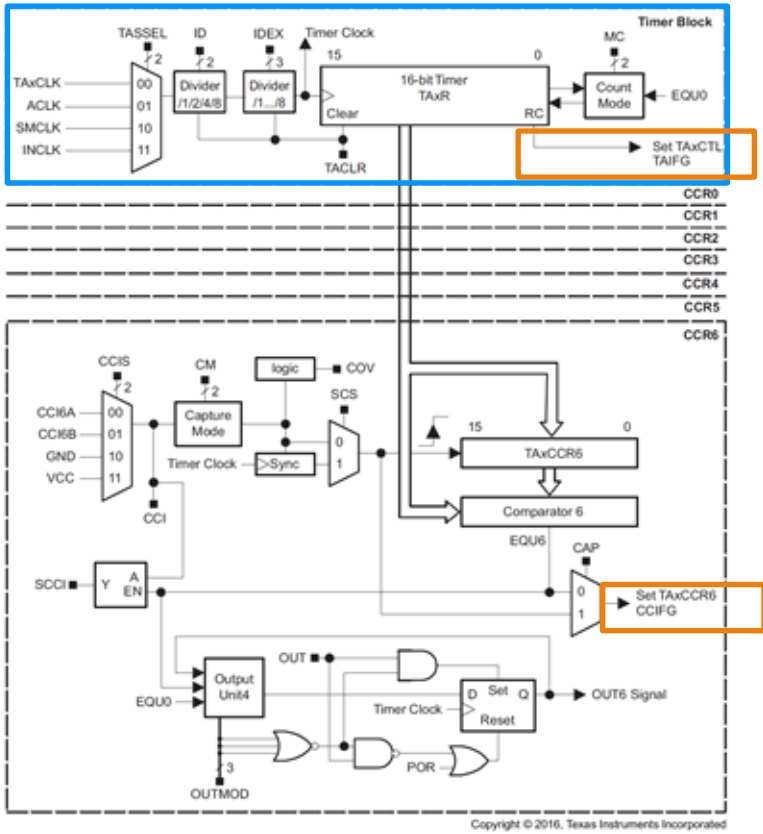


Figure 17-1. Timer_A Block Diagram

DriverLib Timers

- TAxCCR0 interrupt vector for TAxCCR0 CCIFG
  - **TA0_0_IRQHandler**
- TAxIV interrupt vector for all other CCIFG flags and TAIFG
  - **TA0_N_IRQHandler**

```
void TA0_N_IRQHandler(void)
{
    Timer_A_clearInterruptFlag(TIMER_A0_BASE);
    GPIO_toggleOutputOnPin(GPIO_PORT_P1, GPIO_PIN0);
}
```

DriverLib Timers

- TAxCCR0 interrupt vector fo
  - **TA0_0_IRQHandler**
- TAxIV interrupt vector for al
  - **TA0_N_IRQHandler**

void Timer_A_clearInterruptFlag (uint32_t *timer*)

Clears the Timer TAIFG interrupt flag

Parameters

timer	is the instance of the Timer_A module. Valid parameters vary from part to part, but can include: ■ **TIMER_A0_BASE** ■ **TIMER_A1_BASE** ■ **TIMER_A2_BASE** ■ **TIMER_A3_BASE**

Returns
None

```
void TA0_N_IRQHandler(void)
{
   Timer_A_clearInterruptFlag(TIMER_A0_BASE);
   GPIO_toggleOutputOnPin(GPIO_PORT_P1, GPIO_PIN0);
}
```

Lab #3: Timers

- Compile and run up-mode timer on Moodle.
- Create a continuous-mode timer!

Lab #4: FSM

- Implement the following FSM with with
  - Each state transition will take place every 1 se

